

German University in Cairo

Mechatronics Engineering (MCTR601)

Wall Climbing RC

Name	ID	Lab Number
Bassam Walid	55-0499	T-32
Styven Hany	55-10067	T-32
Youssef Mohamed	55-1031	T-32

Table of Contents

1.	Project Description.....	3
2.	Methodology	5
2.1	Mechanical design	5
2.2	Electrical design.....	10
2.3	Control	11
2.3.1.	Modeling.....	11
2.3.2.	Analysis.....	11
2.3.3.	Controller Design.....	12
2.4	Programming.....	13
3.	Design Evaluation.....	21
4.	Appendix.....	22

1. Project Description

This project presents a wall-climbing RC that uses a propeller to generate thrust to overcome gravitational force and increase friction between RC tires and wall allowing it to move freely on walls like it does on the ground. The RC is also 4 wheel driving to help with the initial transition from ground to walls.

The RC is a prototype but with more technology and funding it can be used in several applications including painting, cleaning buildings and checking for cracks using X-ray in any structure.

The car is controller by mobile remote controller but using machine learning it can be taught to navigate buildings on its own without human interaction.

The prototype designed was built to navigate walls of different inclinations using a gyroscope and PID equations that adjusts the propellers direction to overcome forces so the RC can balance when stationary and move normally when needed.

The brushless motor rotating the propeller was adjusted to rotate at certain power and RPM to provide the necessary force along with the tilt.

The RC can be controlled by any mobile phone using a proper application that sends data to a Bluetooth module that communicates with the microcontroller to control the wheels' movement in all 4 directions as well as turning the propeller on and off.

All programming was done on Arduino IDE but on an STM32 Blue Pill microcontroller. STM32 Cube IDE was not used due to multiple errors arising while trying to initialize the brushless motor which could not be solved using built in libraries or commonly used functions of GitHub.

Figure 1 shows all components used. All connections have been tested and shown to work with proper handling of variations and errors that arise from different components such as different gyroscope sensitivities that needs calibrations. All this was handled using proper coding conventions and a lot of testing.

Part Number	Part Name
1	STM32F103C8T6 (Blue Pill)
2	RoaringTop Lipo Battery 2200mAh 11.1V 3S 35C
3	ESC 30A
4	Breadboard
5	Brushless Motor A2212/13T 1400KV
6	Propellers 12x4.5
7	MPU 6050
8	HC-05 Bluetooth module
9	Dual H-Bridge Motor Driver L298N

10	9V DC gearbox yellow motor
11	Servo Motor
12	9V Rechargeable battery
13	Bearings
14	Chassis
15	Bluetooth Electronics mobile application

Figure 1: Table of all components used

2. Methodology

2.1 Mechanical design



Figure 2.0: Initial prototype



Figure 2.1: Initial prototype



Figure 2.2: Initial prototype stable on a wall

Figures 2.0 and 2.1 show the initial prototype for the RC made from balsa wood. The prototype functioned but not fully. Movement on the ground was smooth as long as the propeller was not on.

When turned on the propeller bent the chassis causing rod connecting the wheel and motor to bend so the car couldn't move forward. Supports were needed to handle this

Figure 2.2 shows the prototype stable with no aid on the wall using the propeller. No force was applied by anyone in the photo. They were standing there for safety to unhook the Lipo battery in case of any problems.

Another problem was faced because the tires of the wheels didn't provide enough friction. This problem was handled by 3D printing wheels and putting bike tires to increase friction

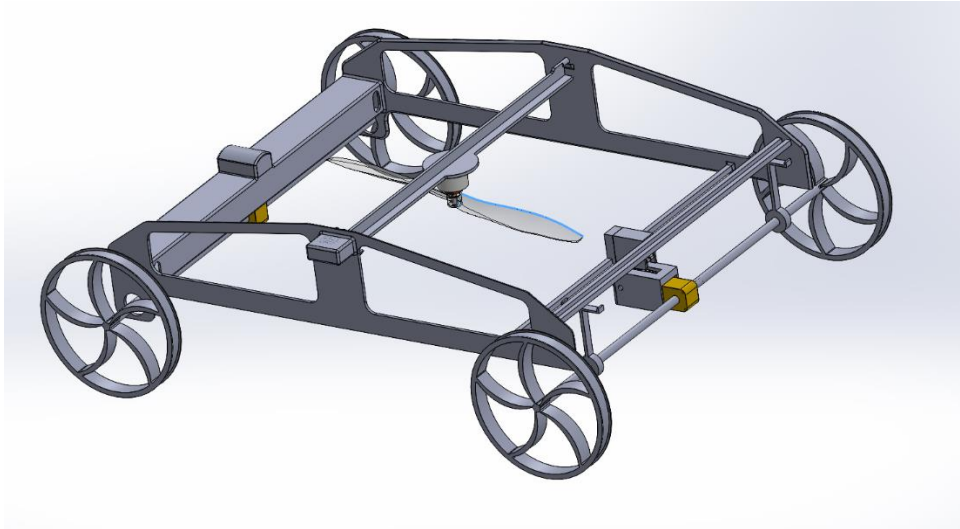


Figure 3.0: Solid Works final design

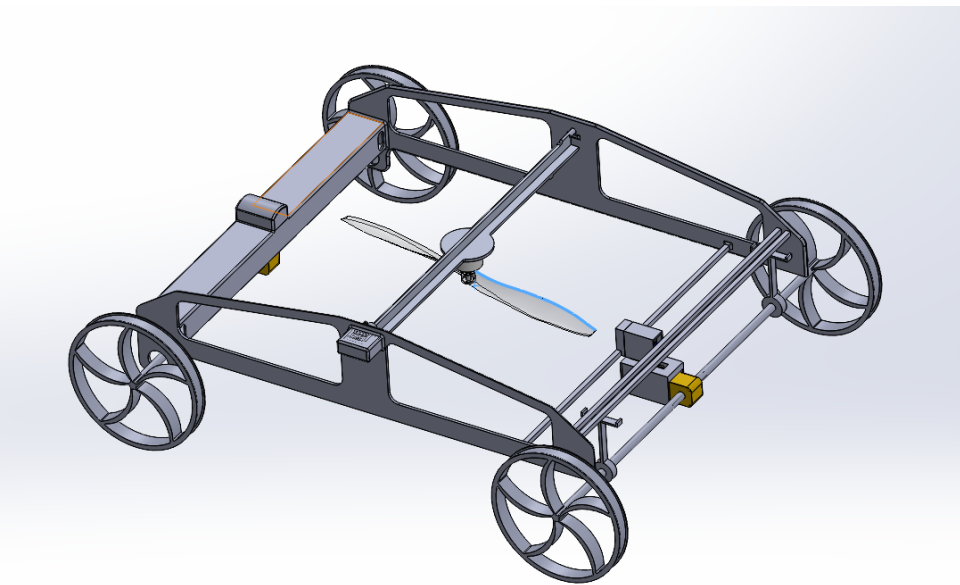


Figure 3.1: Solid Works final design

Figures 3.0 and 3.1 show the solid works model design which was 3D printed using PLA to have low weight and high strength to sustain the downwards force of the propeller

The model was also designed to have housings for all components because it was a major issue with the prototype. Support was also added to prevent the bending mentioned above in the prototype.

The direction of the propeller was reversed to lower the center of mass to have less momentum while climbing walls. All the heavy components were placed at the bottom of the RC to have the same effect.

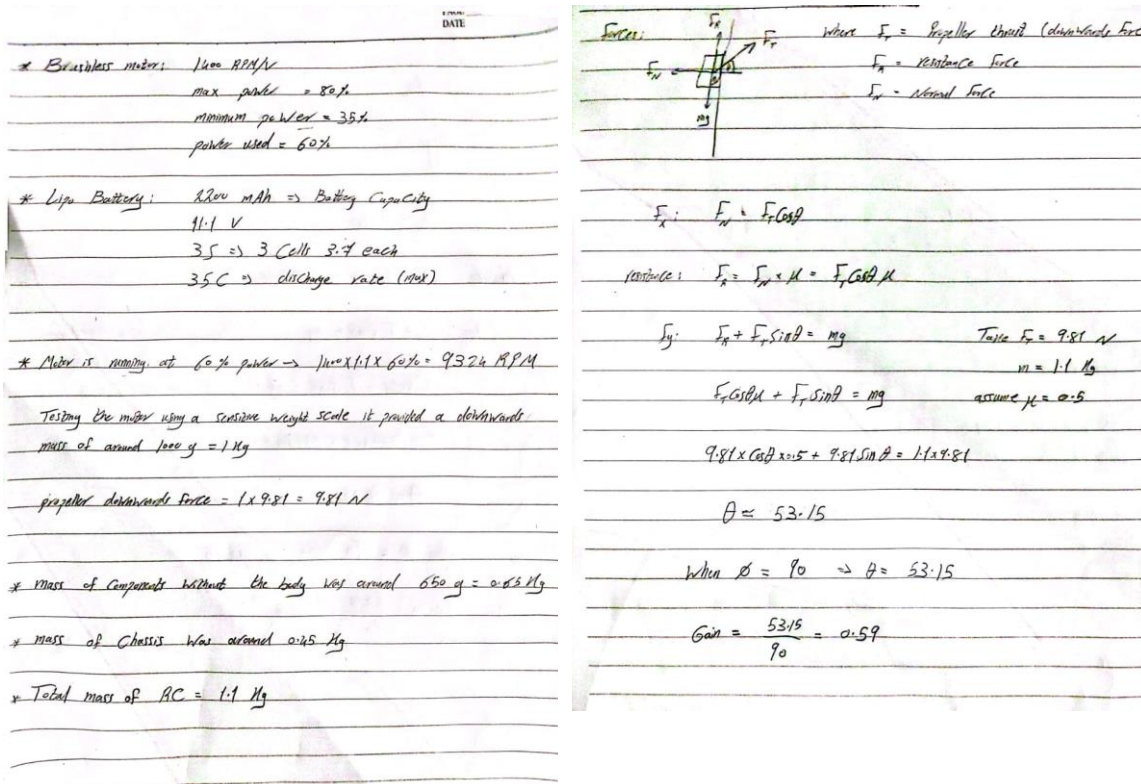


Figure 4: Force Calculations

Figure 4 shows the force calculations used to make sure the propeller provides enough force for the RC to stick to walls of different inclinations

The downwards thrust of the propeller was calculated by placing the brushless motor and the fan on a sensitive weight scale and taking the mass reading.

Problems were faced when trying to choose the friction coefficient as it varies greatly depending on several factors such as material of structure the RC is moving on as well as humidity and speed of the RC.

Upon research, a value of 0.5 was taken as an average and it was further confirmed by testing the vehicle's stability on different structures.

2.2 Electrical design

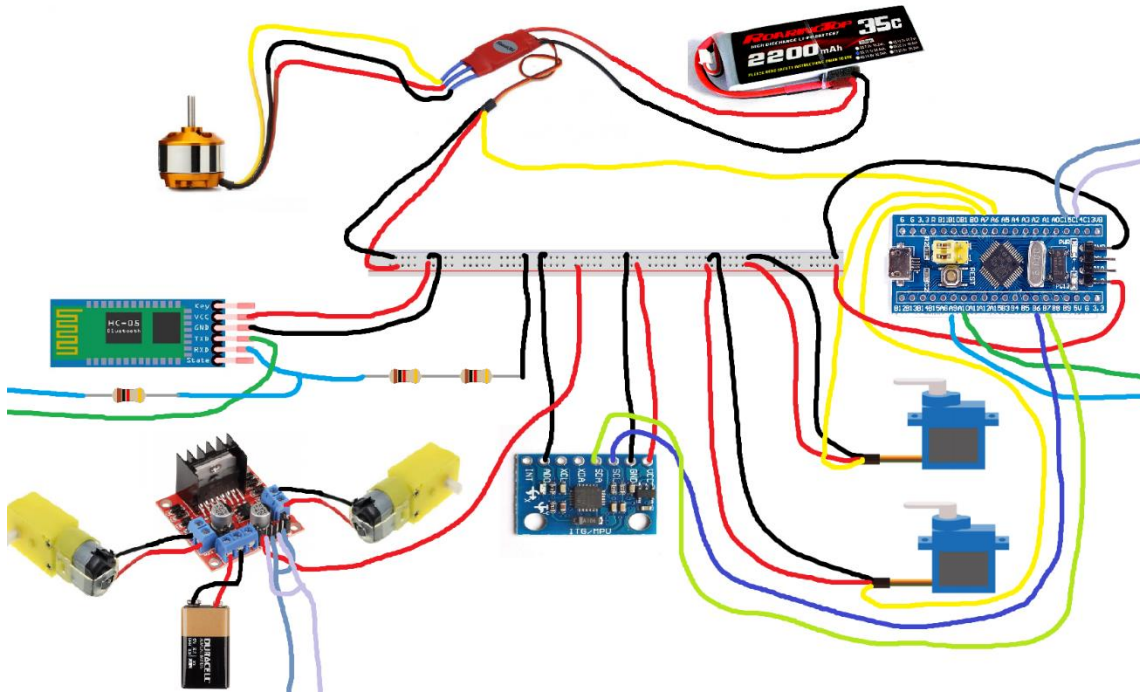


Figure 5: RC Circuit implementation

Firstly, the circuit is powered by a 3s 2200 mAh lipo battery which powers the brushless motor directly and then a 5v is taken from the esc to power the STM, servo motors, Bluetooth module and the imu and another 9v battery is used to power the h bridge and dc motors. three resistors were added to the hc-05 module to the rxd pin because it takes voltage of 3.3 volts while the module itself requires 5v to operate.

2.3 Control

2.3.1. Modeling

Modeling a SG-90 Servo on Simulink can be very challenging so, after researching the transfer function was obtained from the below article:

<https://drive.google.com/file/d/1bEGKZq8Eya3V47Zb28fm3e5QDolXpVxH/view>

2.3.2. Analysis

Open-loop Simulation:

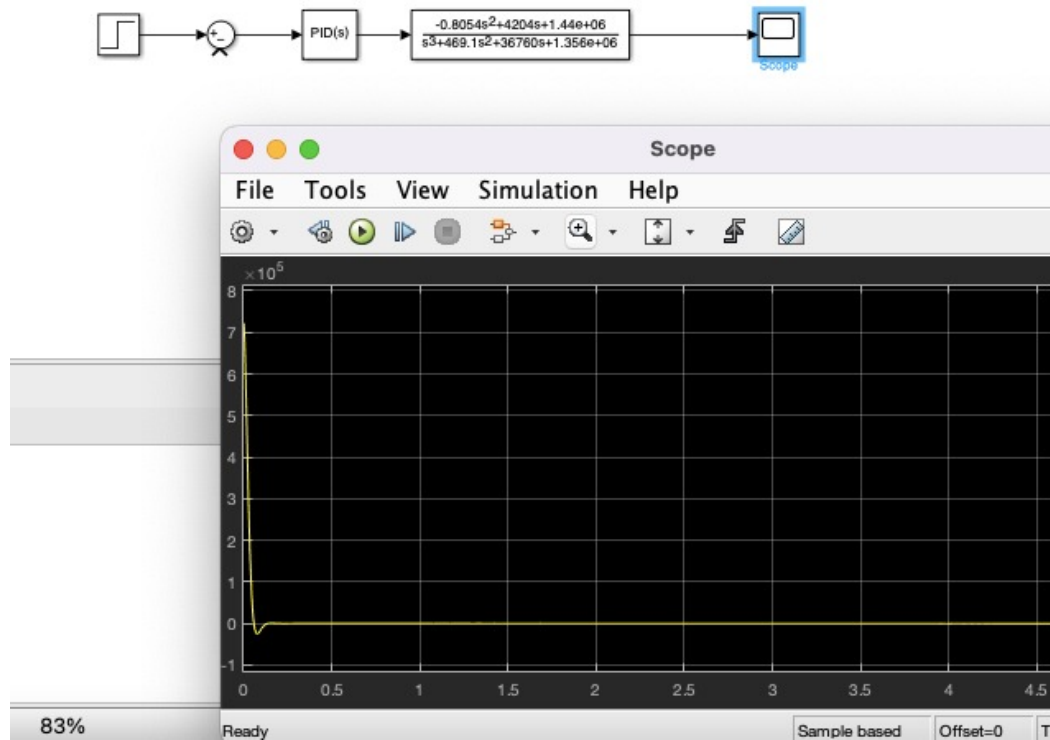


Figure 6.0: Open Loop MatLab Simulation

2.3.3. Controller Design

To get the best response Tune option was used to get the PID values.

Closed loop Simulation:

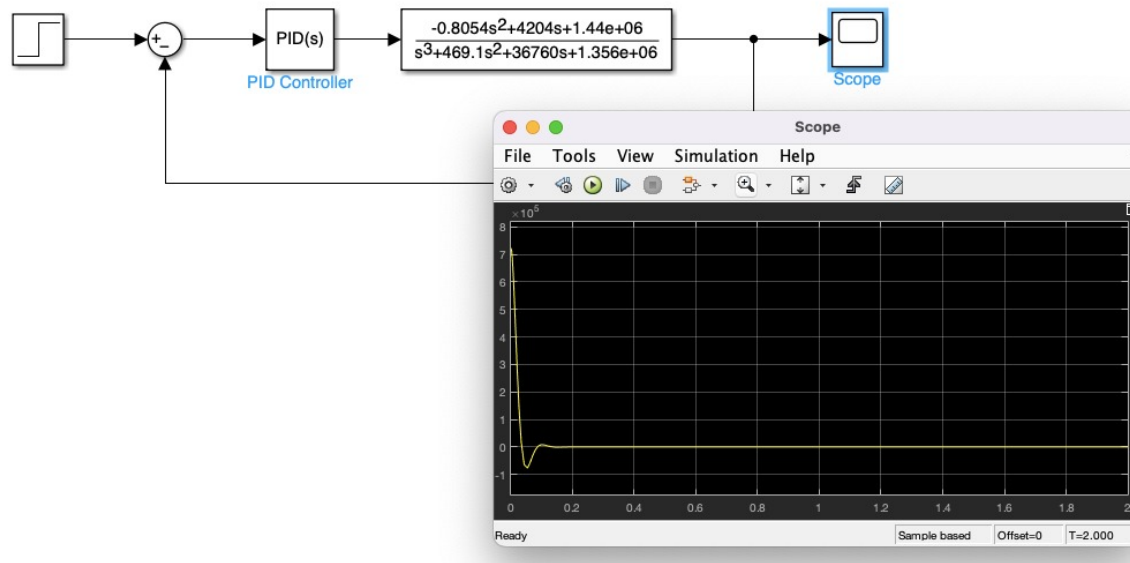


Figure 6.1: Closed Loop MatLab Simulation

2.4 Programming

All programming was done on Arduino IDE by connecting it to STM32 Blue Pill using the library “https://dan.drown.org/stm32duino/package_STM32duino_index.json”

Code:

```
#include <MapleFreeRTOS821.h>

#include <Servo.h>

#include <SoftWire.h>

#include <Wire.h>


Servo ESC;

int speed;

float RateRoll, RatePitch, RateYaw;

float AccX, AccY, AccZ;

float AngleRoll, AnglePitch;

Servo servo1;

Servo servo;

int servoAngle;

float Kp = 0.59; //gain

char data = 0;


void servoAngleBrushless(float angle){ //direction L = left, R = right

    servoAngle = (int) angle * Kp;

    servoAngle = servoAngle +106 ;

    servo.write(servoAngle);

}

void PID_Controller_servo(void *pvParameters);

void Bluetooth(void *pvParameters);


void initializeServo(){
```

```

servo1.attach(PA7);
}

void servo1Angle(char direction){ //direction L = left, R = right
    if(direction == 'L'){
        servo1.write(135);

    }else if(direction == 'R'){
        servo1.write(45);
    }
    else{
        servo1.write(90);
    }
}

void initializeMotor(){
    pinMode(PC14,OUTPUT);
    pinMode(PC15,OUTPUT);
}

void setSpeed(int speed){ //set brushless motor speed from 1 to 100

    int brushlesssspeed = map(speed, 0, 100, 0, 180); //map values 0 -> 0, 100 -> 180
    ESC.write(brushlesssspeed);
}

void initilaizeESC(){
    ESC.attach(PA6); //Adds ESC to pin A6 to output PWM.
    setSpeed(0); //intialise ESC needs an input of 0 for 1 second
    delay(1000);
}

```

```

void BrushlessSpeed(int motorSpeed){ //motor speed should not be less than 35 or more than 85

    for(speed = 0; speed <= motorSpeed; speed += 5) { //Speed is equal to power percentage, min = 35%,
max = 85%

        setSpeed(speed); //Creates variable for speed to be used in in for loop

        delay(500);

    }

}

void initializeServoBrushless(){

    servo.attach(PB0);

}

void gyroInitialize(){

    //Serial1.begin(9600);

    Wire.setClock(400000);

    Wire.begin();

    delay(250);

    Wire.beginTransmission(0x68); //configure Gyroscope output

    Wire.write(0x6B);

    Wire.write(0x00);

    Wire.endTransmission();

}

void gyro_signal(void){

    Wire.beginTransmission(0x68); //switch on low pass filter

    Wire.write(0x1A);

    Wire.write(0x06);

    Wire.endTransmission();

    Wire.beginTransmission(0x68); //configure accelerometer output

    Wire.write(0x1C);

    Wire.write(0x10);

```

```

Wire.endTransmission();

Wire.beginTransmission(0x68); //Pull accelermotor measurments from sensor

Wire.write(0x3B); //Register that reads AccX raw data first 8 bits, the next register reads the last 8 bits
and the ones after does the same for Y and Z

Wire.endTransmission();

Wire.requestFrom(0x68,6); //reads 6 registers

int16_t AccXLSB = Wire.read() << 8 | Wire.read();
int16_t AccYLSB = Wire.read() << 8 | Wire.read();
int16_t AccZLSB = Wire.read() << 8 | Wire.read();

AccX = (float)AccXLSB/4095 - 0.05; //convert measurments to physical values and add calibration value
AccY = (float)AccYLSB/4095;
AccZ = (float)AccZLSB/4095 + 0.05;

AnglePitch = abs(atan(AccY/sqrt(AccX*AccX + AccZ*AccZ))*1/(3.142/180)); //Calculate the absolute
angles
}

void initializeBluetooth(){
    Serial1.begin(9600); //9600 is baud rate for bluetooth module
}

void setup() {
    initilaizeESC(); //brushlesss initialization must be put first
    //BrushlessSpeed(60);
    gyroInitialize();
    initializeServo();
    initializeServoBrushless();
    initializeMotor();
    initializeBluetooth();
}

```

```

xTaskCreate(PID_Controller_servo, "Task1", 128, NULL, 1, NULL);

xTaskCreate( Bluetooth, "Task2", 128, NULL, 1, NULL);

vTaskStartScheduler();

}


void loop() {


}


void PID_Controller_servo(void *pvParameters) {

    for(;;){

        uint64_t i;

        if(i==1844000000000000000){

            gyroInitialize();

            i=0;

        }

        gyro_signal();

        //Serial1.println(AnglePitch);

        servoAngleBrushless(AnglePitch);

        i++;

        delay(5);

    }

}


void Bluetooth(void *pvParameters) {

    for(;;){

        //if (xQueueReceive(blueetoothQueue, &data, portMAX_DELAY) == pdPASS) {

            if(Serial1.available() > 0){

                char data = Serial1.read();

            }

        }

    }

}

```

```
if (data == 'Q'){  
    BrushlessSpeed(60);  
}  
  
else if (data == 'q'){  
    BrushlessSpeed(0);  
}  
  
else if(data == 'F'){ //forwards  
    digitalWrite(PC14, HIGH);  
    digitalWrite(PC15, LOW);  
}  
  
else if(data == 'B'){ //backwards  
    digitalWrite(PC14, LOW);  
    digitalWrite(PC15, HIGH);  
}  
  
else if(data == 'L'){ //left  
    servo1Angle('L');  
}  
  
else if(data == 'R'){ //right  
    servo1Angle('R');  
}  
  
else if(data == 'A'){ //forward right  
    digitalWrite(PC14, HIGH);  
    digitalWrite(PC15, LOW);
```

```
servo1Angle('R');
}

else if(data == 'B'){ //backwards right
    digitalWrite(PC14, LOW);
    digitalWrite(PC15, HIGH);
    servo1Angle('R');
}

else if(data == 'C'){ //backwards left
    digitalWrite(PC14, LOW);
    digitalWrite(PC15, HIGH);
    servo1Angle('L');
}

else if(data == 'D'){ //forward left
    digitalWrite(PC14, HIGH);
    digitalWrite(PC15, LOW);
    servo1Angle('L');
}

else if(data == 'E'){ //brake
    digitalWrite(PC14, HIGH);
    digitalWrite(PC15, HIGH);
    servo1Angle('E');
}

else if(data == 'Y'){
    initializeServoBrushless();
    servoAngleBrushless(AnglePitch);
```

```
    }  
    gyroInitialize();  
  }  
  //delay(5); }  
}  
}
```

3. Design Evaluation

The first experiment was made with a prototype made of balsa wood we started by cutting balsa wood to the desired shapes and then glued the needed pieces together with a glue gun. Upon the first trial it was clear that the model was not able to handle the downforce of the propeller, as the axles of the wheels bent as the brushless motor was turned on, but it was still able to stabilize itself on the wall without any assistance as shown in *figure 2.2*

the experiment backed up our theoretical calculations and equations shown in *figure 5* based on our first experiment we decided to 3D print our final model to make it stronger and lighter and we added supports to the wheel axles to eliminate the problem we faced in the first model. The material we used for 3D printing was PLA as it has high strength, low weight and is economically efficient.

4. Appendix

Brushless motor

MODEL	KV (rpm/V)	Voltage (V)	Prop	Load Current(A)	Power (W)	Pull (g)	Efficiency (g/W)	Lipo Cell	Weight (g)
A2212	930	11.1	1060	9.8	109	660	6.1	2-4S	52
	1000		1047	15.6	173	885	5.1		
	1400		9050	19.0	210	910	4.3		
	1800		8060	20.8	231	805	3.5		
	2200		6030	21.5	239	732	3.1	2-3S	
	2450		6030	25.2	280	815	2.9		

Propellers

- Weight: 13G+/-PC
- Rotation: Clockwise / Counter Clockwise (CW/CCW)
- Width: 12 inches
- Pitch: 4.5 inches
- Diameter of shaft: 6.0mm
- Thickness of center: 9.7mm
- Recommended wheelbase: 550mm – 700mm
- Recommended motor: 800KV – 1100KV

30A Brushless ESC

- Output: Continuous 30A, burst 40A up to 10 Secs.
- Input voltage: 2-4 cells lithium battery or 5-12 cells NiCd/NIMh battery
- BEC: 2A / 5V (Linear mode).
- Max speed: 210,000rpm for 2 poles BLM, 70,000rpm for 6 poles BLM, 35,000rpm for 12 poles BLM.
- Size: 45 * 24 * 11mm / 1.8 * 0.9 * 0.4in
- Weight: 25g / 0.9oz
- Item total weight: 480g / 1.06Lbs